

Diseño e Implementación de un Mini Monitor de Recursos para Linux Basado en Python

Danny Ayuquina, Saray Canarte, y Domenica Villagomez

Carrera de Ingeniería de Software

Universidad de las Fuerzas Armadas ESPE, Ecuador

Correo: dmayuquina@espe.edu.ec, sacanarte@espe.edu.ec, dnvillagomez@espe.edu.ec

Este artículo describe el diseño, la implementación y la validación de un monitor de recursos para sistemas Linux desarrollado con Python. El objetivo principal fue aplicar de forma práctica conceptos fundamentales de Sistemas Operativos, entre ellos la lectura del sistema de archivos virtual `proc`, la creación de procesos mediante la llamada `fork`, la ejecución concurrente con hilos y la administración de comandos del sistema. La aplicación permite observar en tiempo real el estado de la CPU, la memoria, el disco, los procesos activos, los usuarios conectados y las interfaces de red, además de gestionar un historial de capturas mediante operaciones de creación, consulta, actualización y eliminación sobre una base de datos SQLite. Los resultados muestran que la herramienta reproduce con precisión los valores reportados por utilidades estándar de Linux como `free`, `df`, `ps` y `who`, y que la combinación de un proceso hijo con hilos internos permite capturar el estado del sistema sin bloquear la interfaz principal. Se concluye que el proyecto cumple con los requerimientos funcionales y técnicos establecidos y que constituye una base sólida para futuras extensiones.

Palabras clave—Sistemas operativos, Linux, monitoreo de recursos, procesos, hilos, sistema de archivos `proc`, Python, SQLite.

I. INTRODUCCIÓN

Los sistemas operativos modernos exponen mecanismos que permiten conocer el estado interno de los recursos que administran, entre ellos el procesador, la memoria, el almacenamiento y la red. En Linux, gran parte de esta información se obtiene a través del sistema de archivos virtual `proc`, un conjunto de archivos generados dinámicamente por el núcleo que reflejan el estado del sistema en cada instante. Herramientas ampliamente conocidas como `top`, `free` o `df` construyen su funcionalidad precisamente sobre esta fuente de datos.

Comprender cómo se obtiene y se procesa esta información resulta central en la formación de un ingeniero de software, ya que involucra conceptos estudiados en la asignatura de Sistemas Operativos como la gestión de procesos, la concurrencia mediante hilos, la comunicación entre procesos y la interacción directa con llamadas del sistema. Muchas aplicaciones comerciales de monitoreo, entre ellas `htop` o `glances`, ocultan esta complejidad detrás de bibliotecas de alto nivel como `psutil`. Este proyecto tomó una ruta distinta de manera intencional, evitando bibliotecas que abstraigan el acceso al sistema operativo, con el fin de que cada dato mostrado en pantalla provenga de una lectura directa del núcleo o de un comando estándar de Linux.

Con este propósito se desarrolló el Mini Monitor de Recursos, una aplicación de consola escrita en Python que integra estos conceptos en un producto funcional y verificable. La aplicación combina cuatro elementos que suelen estudiarse por separado en el curso: la lectura del sistema de archivos `proc`, la creación de procesos mediante `fork`, la concurrencia mediante

hilos, y la persistencia de información en una base de datos relacional accesible mediante operaciones CRUD.

El presente artículo documenta el proceso de desarrollo del proyecto. La sección II describe la metodología empleada. La sección III detalla la arquitectura y la implementación de cada componente. La sección IV presenta los resultados obtenidos durante las pruebas de verificación. Finalmente, la sección V expone las conclusiones del trabajo.

II. METODOLOGÍA

El desarrollo del proyecto siguió un enfoque incremental distribuido en cinco semanas, de acuerdo con el cronograma establecido por la asignatura. Durante la primera semana se conformó el equipo de trabajo, se analizaron los requerimientos funcionales y técnicos, y se diseñó la arquitectura general orientada en capas junto con el esquema de la base de datos. En la segunda semana se implementaron los módulos de CPU y memoria a partir de la lectura directa de archivos en `proc`. La tercera semana se dedicó a los módulos de disco, red, procesos y usuarios conectados, apoyados en comandos del sistema operativo. Durante la cuarta semana se incorporó la concurrencia mediante procesos hijos e hilos, y se completaron las operaciones de creación, consulta, actualización y eliminación. La quinta semana comprendió la integración final, las pruebas de verificación y la elaboración de la documentación.

Como herramientas de trabajo se utilizaron Visual Studio Code como entorno de desarrollo, Python 3 como lenguaje de programación, Git y GitHub para el control de versiones. Dado que las funcionalidades requeridas dependen de características exclusivas del núcleo de Linux, se utilizó una máquina virtual con Ubuntu Desktop como entorno nativo de ejecución.

Para facilitar el desarrollo entre diferentes sistemas operativos, se configuró el adaptador de red de la máquina virtual en modo NAT, con una regla de redirección del puerto 22,

correspondiente al servicio SSH, hacia el puerto 2222 del equipo anfitrión, y se instaló y activó el servicio de servidor SSH mediante el paquete openssh-server. Sobre esta conexión se empleó la extensión Remote-SSH de Visual Studio Code. Esta herramienta vincula el entorno de desarrollo del equipo anfitrión directamente con el sistema de archivos y la terminal de la máquina virtual. Su uso permitió editar el código fuente desde el sistema principal mientras la aplicación se ejecutaba y evaluaba de forma nativa dentro de Ubuntu. Bajo este esquema, el monitor de recursos operó estrictamente como un proceso local en el entorno Linux, garantizando el acceso a los archivos del directorio `/proc` y a las utilidades del sistema sin requerir la implementación de componentes de red adicionales en el código.

Para la interfaz de usuario se empleó la biblioteca Textual, la cual renderiza de forma interactiva los datos recolectados directamente sobre la terminal expuesta por la extensión. Para la persistencia de datos se seleccionó SQLite por su simplicidad de instalación y su adecuación a un proyecto académico de este alcance.

El trabajo se organizó de forma colaborativa entre los tres integrantes del equipo, quienes se repartieron los módulos según su afinidad técnica y coordinaron la integración final mediante revisiones conjuntas del repositorio en GitHub.

La verificación del sistema se realizó de manera manual, comparando los valores reportados por la aplicación con la salida de comandos estándar de Linux equivalentes, entre ellos `free`, `df`, `ps` y `who`. También se verificó el comportamiento de la concurrencia observando los identificadores de proceso generados por `fork` y confirmando que el proceso padre esperaba correctamente la finalización del proceso hijo antes de continuar. Adicionalmente, cada módulo incluyó un bloque de prueba ejecutable de forma independiente, lo que permitió validar su comportamiento antes de integrarlo con el resto de la aplicación.

III. DESARROLLO

A. Arquitectura general

El sistema se organizó en capas con responsabilidades claramente separadas, siguiendo un esquema similar al de una arquitectura en capas convencional. La Figura 1 presenta el diagrama de componentes del sistema, y la Tabla I resume la responsabilidad de cada capa.

Esta separación permite que cada capa pueda modificarse de forma independiente. Por ejemplo, la fuente de los datos del sistema podría cambiarse sin afectar la interfaz de usuario, ya que esta última solo depende de la capa de servicios.

B. Lectura del sistema de archivos `proc`

La información de CPU y memoria se obtiene leyendo directamente los archivos `proc/cpuinfo`, `proc/meminfo` y `proc/loadavg`. El número de núcleos se calcula contando las líneas que inician con la palabra `processor`, mientras que la frecuencia se obtiene del primer valor reportado en el campo `cpu MHz`. La carga promedio del sistema se lee del primer valor de `proc/loadavg`. Para la memoria se procesa cada línea

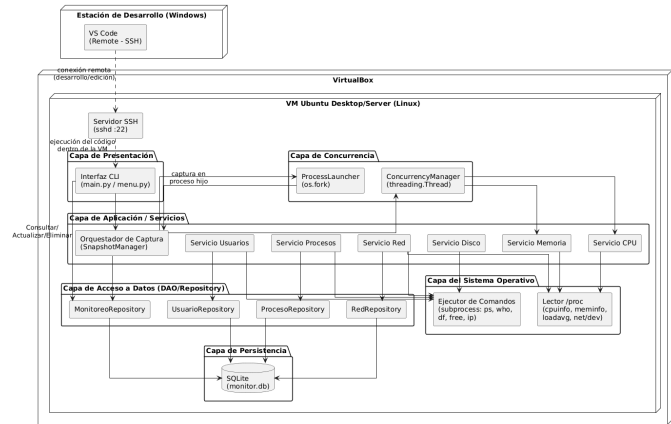


Fig. 1. Arquitectura en capas del Mini Monitor de Recursos.

TABLA I
CAPAS DE LA ARQUITECTURA DEL SISTEMA

Capa	Responsabilidad
Presentación	Interfaz de texto interactiva construida con Textual, organizada en pestañas.
Servicios	Unifica el acceso a los datos del sistema y los expone a las capas superiores.
Concurrencia	Orquesta la captura de datos mediante un proceso hijo y dos hilos internos.
Repositorio	Ejecuta las operaciones de creación, consulta, actualización y eliminación sobre SQLite.
Sistema operativo	Lee archivos de <code>proc</code> y ejecuta comandos del sistema mediante <code>subprocess</code> .

de `proc/meminfo`, y la memoria usada se calcula como la diferencia entre la memoria total y la memoria disponible.

Las estadísticas de red se obtienen de `proc/net/dev`, un archivo que reporta, por cada interfaz, el número de bytes y paquetes recibidos y enviados desde que el sistema fue iniciado. El Listado 1 muestra de forma simplificada la lectura de `proc/cpuinfo` y `proc/loadavg`.

```
def get_cpu_info():
    nucleos = 0
    frecuencia = 0.0
    with open("/proc/cpuinfo") as f:
        for linea in f:
            if linea.startswith("processor"):
                nucleos += 1
            if linea.startswith("cpu MHz") and frecuencia == 0.0:
                frecuencia = float(linea.split(":")[1])
    with open("/proc/loadavg") as f:
        carga = float(f.read().split()[0])
    return {"nucleos": nucleos,
            "frecuencia_mhz": frecuencia,
            "carga_1min": carga}
```

Listado 1. Lectura de información de CPU desde `proc`

Este enfoque evita depender de bibliotecas externas para obtener información que el núcleo ya expone de forma directa, lo que refuerza el objetivo académico del proyecto de trabajar

con las fuentes primarias del sistema operativo.

C. Ejecución de comandos del sistema

Para la información que no está disponible directamente en `proc`, se ejecutan comandos estándar de Linux mediante el módulo `subprocess`. El comando `df` se utiliza para obtener el espacio total, usado y libre del disco. El comando `ps` se utiliza para listar los procesos activos junto con su identificador, nombre, estado y usuario propietario. El comando `who` se utiliza para obtener la lista de usuarios conectados y su tiempo de conexión.

En todos los casos se utilizó `subprocess.run` con captura de salida en modo texto, en lugar de `os.system`, ya que este enfoque permite procesar el resultado del comando de forma estructurada dentro de Python sin depender de redirecciones hacia archivos intermedios. Cada función de esta capa devuelve directamente una lista de diccionarios o un diccionario simple, de manera que las capas superiores no necesitan conocer el formato original de la salida del comando.

D. Concurrencia mediante procesos e hilos

Uno de los requisitos técnicos centrales del proyecto consistía en aplicar procesos e hilos de forma conjunta. Cada vez que el usuario solicita una nueva captura del sistema, la aplicación crea un proceso hijo mediante la llamada `fork`. Dentro de ese proceso hijo se lanzan dos hilos independientes, uno encargado de leer el estado de la CPU y otro encargado de leer el estado de la memoria, ambos ejecutándose en paralelo mediante la clase `Thread` del módulo `threading`.

Dado que `fork` genera una copia independiente del espacio de memoria del proceso, el resultado obtenido en el hijo no es visible automáticamente para el proceso padre. Para resolver esta comunicación se utilizó un mecanismo de tubería, mediante el cual el hijo escribe el identificador del registro recién creado y el padre lo lee después de esperar la finalización del hijo con la llamada `waitpid`. Este diseño evita que la interfaz principal quede bloqueada mientras se ejecuta el resto del programa. El Listado 2 representa esta lógica.

```
def crear_snapshot_en_proceso_hijo(comentario, etiqueta):
    :
    lector, escritor = os.pipe()
    pid = os.fork()

    if pid == 0:
        os.close(lector)
        datos = _capturar_con_hilos()
        nuevo_id = crear_monitoreo(
            datos["cpu"], datos["mem"],
            datos["disco"], comentario, etiqueta)
        os.write(escritor, str(nuevo_id).encode())
        os._exit(0)
    else:
        os.close(escritor)
        os.waitpid(pid, 0)
        resultado = os.read(lector, 100).decode()
        return int(resultado)
```

Listado 2. Creación de proceso hijo con hilos internos

TABLA II
ESQUEMA DE LA BASE DE DATOS

Tabla	Contenido
monitoreos	Fecha, métricas de CPU, memoria y disco, comentario y etiqueta de cada captura.
procesos	PID, nombre, estado y usuario de cada proceso activo en la captura.
usuarios_conectados	Usuario y tiempo de conexión asociados a la captura.
interfaces_red	Nombre, dirección IP, bytes y paquetes de cada interfaz en la captura.

Cada llamada a esta función genera un proceso hijo de vida corta, cuyo único propósito es capturar el estado del sistema en un instante determinado y almacenarlo en la base de datos. Al finalizar su tarea, el hijo termina mediante `os._exit`, evitando así ejecutar de nuevo el código posterior del proceso padre, un error común al trabajar con `fork` en Python.

E. Persistencia y operaciones CRUD

Los datos capturados se almacenan en una base de datos SQLite compuesta por cuatro tablas relacionadas. La tabla `monitoreos` guarda la captura principal, incluyendo fecha, métricas de CPU, memoria y disco, además de un comentario y una etiqueta definidos por el usuario. Las tablas `procesos`, `usuarios_conectados` e `interfaces_red` almacenan los datos detallados asociados a cada captura, vinculados mediante una clave foránea con eliminación en cascada.

Sobre esta base de datos se implementaron las cuatro operaciones CRUD requeridas. La operación de creación guarda una captura completa junto con todos sus datos relacionados. La operación de consulta permite listar todas las capturas o revisar el detalle completo de una en particular. La operación de actualización permite modificar el comentario o la etiqueta de una captura ya existente. La operación de eliminación borra una captura y, gracias a la restricción de eliminación en cascada configurada en el esquema, elimina automáticamente sus registros asociados. La Tabla II describe las cuatro tablas que conforman el esquema.

Cada conexión a la base de datos habilita explícitamente la restricción PRAGMA `foreign_keys`, ya que SQLite no aplica las llaves foráneas de forma predeterminada. Sin esta configuración, la eliminación en cascada definida en el esquema no tendría efecto alguno.

F. Interfaz de usuario

La interfaz se organiza en cinco pestañas: Resumen, Procesos, Usuarios, Red e Historial. Las primeras cuatro se actualizan automáticamente cada dos segundos mediante el método `set_interval` de `Textual`, mientras que la pestaña Historial se actualiza después de cada operación sobre la base de datos. La actualización periódica se ejecuta en el hilo principal de la interfaz, por lo que las lecturas del sistema deben ser suficientemente rápidas para no afectar la fluidez de la aplicación, algo que se confirmó durante las pruebas. La Tabla III resume los atajos de teclado disponibles.

TABLA III
ATAJOS DE TECLADO DE LA APLICACIÓN

Tecla	Acción
r	Actualiza manualmente los datos en pantalla.
c	Crea una nueva captura del sistema.
v	Muestra el detalle completo de la captura seleccionada.
e	Edita el comentario o la etiqueta de la captura seleccionada.
d	Elimina la captura seleccionada.
q	Cierra la aplicación.

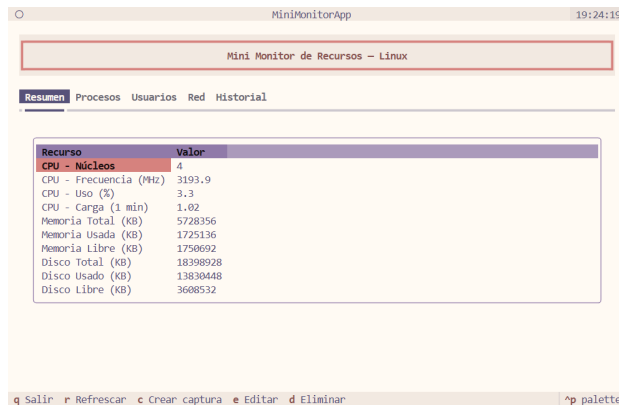


Fig. 2. Pestaña Resumen con los valores de CPU y memoria leídos desde proc.

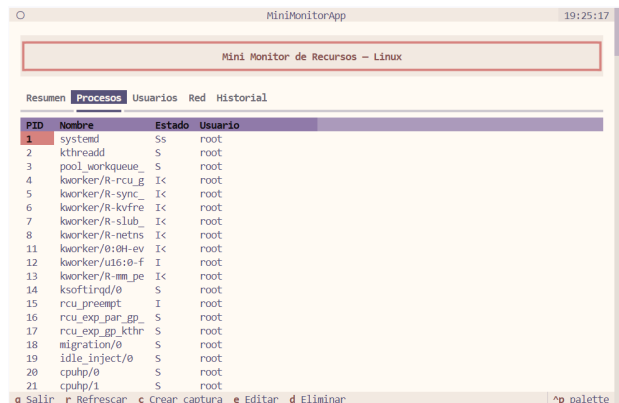


Fig. 3. Pestaña Procesos con la lista de procesos activos obtenida mediante ps.

La Figura 2 muestra la pestaña Resumen en ejecución, con los valores de CPU y memoria obtenidos directamente de proc, mientras que la Figura 3 presenta la pestaña Procesos, donde cada fila corresponde a un proceso activo obtenido con el comando ps.

Las pestañas Usuarios y Red siguen el mismo formato tabular, mostrando respectivamente las sesiones activas obtenidas mediante who y las estadísticas de tráfico e IP de cada interfaz obtenidas desde proc/net/dev y el comando ip.

Adicionalmente, la operación de creación y la de actualización cuentan con cuadros de diálogo propios dentro de la interfaz. La Figura 4 presenta ambos cuadros: el de la izquierda corresponde a la creación de una nueva captura, y el de la

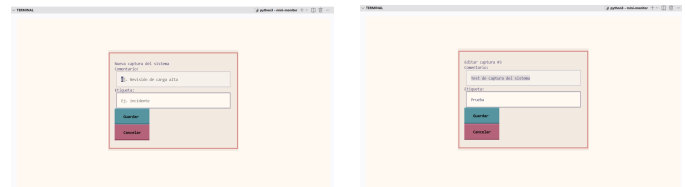


Fig. 4. Cuadros de diálogo para crear (izquierda) y editar (derecha) una captura.



Fig. 5. Vista de detalle de una captura almacenada, mostrando los procesos, usuarios e interfaces de red asociados.

derecha a la edición del comentario y la etiqueta de una captura ya existente.

Por su parte, la operación de consulta permite revisar el detalle completo almacenado para una captura específica, incluyendo los procesos, usuarios e interfaces de red registrados en el momento de la captura, tal como se muestra en la Figura 5.

IV. RESULTADOS

Durante la etapa de verificación se compararon los valores mostrados por el Mini Monitor con la salida de los comandos equivalentes del sistema. Los valores de memoria coincidieron con los reportados por free, los valores de disco coincidieron con df, y la lista de procesos y usuarios conectados coincidió con la salida de ps y who respectivamente. Esto confirma que la lectura de proc y la ejecución de comandos se realizó correctamente.

Con respecto a la concurrencia, se verificó que cada captura generaba un identificador de proceso distinto al del proceso principal, confirmando que fork efectivamente creaba un proceso independiente. También se comprobó que el proceso padre esperaba la finalización completa del hijo antes de continuar, evitando así condiciones de carrera al momento de leer el resultado de la captura. La Figura 6 muestra esta prueba ejecutada directamente en la terminal, incluyendo el identificador de proceso generado y una consulta posterior a la base de datos que confirma que la captura quedó almacenada correctamente.

Las cuatro operaciones CRUD se probaron de forma secuencial. Se creó una captura de prueba, se consultó su contenido completo, se actualizó su comentario y su etiqueta, y finalmente se eliminó, confirmando en cada paso que la base de datos reflejaba correctamente el cambio realizado. La Tabla IV resume los casos de prueba ejecutados junto con su resultado.

Todos los casos de prueba mostrados en la Tabla IV se ejecutaron de forma manual sobre una máquina virtual con

```

dnavillagomez@ubuntu-desktop-VirtualBox:~$ cd ProyectoFinal
dnavillagomez@ubuntu-desktop-VirtualBox:~/ProyectoFinal$ cd Mini-Monitor
dnavillagomez@ubuntu-desktop-VirtualBox:~/ProyectoFinal/Mini-Monitor$ sqlite3
db/monitor.db "SELECT id, fecha_hora, comentario, etiqueta FROM monitoreos;"
id|fecha_hora|comentario|etiqueta
1|2026-07-26T13:37:21|Snapshot vía fork+threading|fork_test
2|2026-07-26T20:49:58|Snapshot vía fork+threading|fork_test
dnavillagomez@ubuntu-desktop-VirtualBox:~/ProyectoFinal/Mini-Monitor$ █

```

Fig. 6. Prueba en terminal de la creación de un snapshot mediante fork y threading, seguida de una consulta SQL que confirma el registro almacenado.

TABLA IV
CASOS DE PRUEBA EJECUTADOS

Caso de prueba	Resultado observado
Lectura de CPU y memoria contra free y top	Valores coincidentes
Lectura de disco contra df	Valores coincidentes
Lista de procesos contra ps	Coincidencia en PID y estado
Lista de usuarios contra who	Coincidencia total
Creación de proceso hijo con fork	PID del hijo distinto al del padre
Espera del padre con waitpid	Sin condiciones de carrera
Creación de una captura (CRUD)	Registro insertado correctamente
Consulta de una captura (CRUD)	Datos relacionados recuperados
Actualización de comentario y etiqueta	Cambios reflejados en la consulta
Eliminación de una captura	Registros hijos eliminados en cascada

TABLA V
CUMPLIMIENTO DE REQUERIMIENTOS FUNCIONALES

Módulo	Estado	Observación
CPU	Completo	Núcleos, frecuencia y carga
Memoria	Completo	Total, usada, libre y swap
Procesos	Completo	PID, nombre, estado, usuario
Usuarios	Completo	Usuario y tiempo de conexión
Disco	Completo	Espacio total, usado y libre
Red	Completo	Interfaz, dirección IP y tráfico
CRUD	Completo	Creación, consulta, edición y borrado

Ubuntu, utilizando los bloques de prueba incluidos en cada módulo del proyecto. Ningún caso presentó un resultado distinto al esperado.

La Tabla V resume el estado de cumplimiento de los requerimientos funcionales establecidos en el documento del proyecto.

Todos los módulos y operaciones definidos en el documento de requerimientos se implementaron en su totalidad,

incluyendo la dirección IP por interfaz de red, obtenida mediante el comando `ip` y combinada con las estadísticas de tráfico leídas desde `/proc/net/dev`.

V. CONCLUSIONES

El desarrollo del Mini Monitor de Recursos permitió aplicar de forma práctica los conceptos centrales de la asignatura de Sistemas Operativos. La lectura directa del sistema de archivos `proc` mostró cómo el núcleo de Linux expone su estado interno sin necesidad de bibliotecas externas. La implementación conjunta de un proceso hijo mediante `fork` y de hilos internos mediante `threading` permitió comprender de manera concreta la diferencia entre concurrencia a nivel de proceso y concurrencia a nivel de hilo, así como los retos de comunicación que surgen entre procesos que no comparten memoria.

El uso de una base de datos SQLite con operaciones CRUD completas permitió además practicar el diseño de esquemas relacionales y el manejo de integridad referencial mediante eliminación en cascada. La arquitectura organizada en capas facilitó la división del trabajo entre los integrantes del equipo y permitió que cada módulo pudiera probarse de forma independiente antes de la integración final.

Como trabajo futuro se propone completar la captura de direcciones IP por interfaz, incorporar manejo de errores más robusto ante fallos de lectura del sistema, y agregar un mecanismo de registro de eventos para auditar las operaciones realizadas sobre el historial de capturas.

REFERENCIAS BIBLIOGRÁFICAS

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.
- [2] W. Stallings, *Operating Systems: Internals and Design Principles*, 9th ed. Boston, MA, USA: Pearson, 2018.
- [3] The Linux Kernel Documentation, "The `/proc` Filesystem." [Online]. Available: <https://www.kernel.org/doc/html/latest/filesystems/proc.html>
- [4] Python Software Foundation, "os — Miscellaneous operating system interfaces," Python 3 Documentation. [Online]. Available: <https://docs.python.org/3/library/os.html>
- [5] Python Software Foundation, "threading — Thread based parallelism," Python 3 Documentation. [Online]. Available: <https://docs.python.org/3/library/threading.html>
- [6] SQLite Consortium, "SQLite Documentation." [Online]. Available: <https://www.sqlite.org/docs.html>
- [7] Textualize Inc., "Textual Documentation." [Online]. Available: <https://textual.textualize.io/>